Buổi 12

Cây cân bằng AVL và Ôn tập tổng hợp

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Cây cân bằng

1.1 Vấn đề suy biến và khái niệm cây cân bằng
1.2 Chỉ số cân bằng và chiều cao cây AVL
1.3 Cấu trúc dữ liệu cây AVL

2. Cân bằng lại cây AVL

2.1 Các trường hợp mất cân bằng
2.2 Quay đơn: quay phải
2.3 Quay kép: LR và RL
2.4 Thêm nút vào cây AVL
2.5 Minh họa thêm dãy khóa vào cây AVL
2.6 Hủy nút trên cây AVL
2.7 Đánh giá cây AVL

3. Ôn tập tổng hợp

3.1 Hệ thống lại các cấu trúc dữ liệu đã học
3.2 Hệ thống lại các giải thuật đã học
3.3 Tiêu chí lựa chọn cấu trúc dữ liệu
3.4 Thực hành buổi 12 – Mini-project và báo cáo so sánh hiệu năng
3.5 Câu hỏi và bài tập ôn tập tổng hợp

1.1 Vấn đề suy biến và khái niệm cây cân bằng

- Buổi 11: mọi thao tác trên BST tốn **O(h)**; nếu thêm dữ liệu **đã có thứ tự**, cây **suy biến** (h = N) ⇒ chậm như DSLK. ⇒ Cần cây **tự điều chỉnh** để chiều cao luôn ≈ $\log_2 N$.

Cây cân bằng hoàn toàn

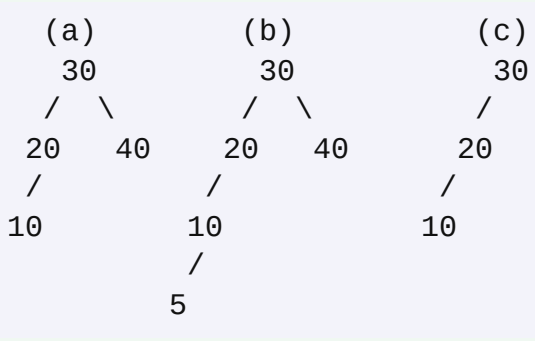
Cây nhị phân mà tại **mọi nút**, **số nút** của cây con trái và cây con phải chênh lệch **không quá 1**.

- Chiều cao nhỏ nhất có thể, nhưng **rất khó duy trì**: một lần thêm có thể buộc phải tổ chức lại toàn bộ cây ⇒ O(N).

Cây cân bằng AVL (Adelson-Velsky & Landis, 1962)

BST mà tại **mọi nút**, **chiều cao** của cây con trái và cây con phải chênh lệch **không quá 1**.

Ví dụ 12.1 – Cây nào là AVL?



- (a): tại 30: h(trái) = 2, h(phải) = 1 ⇒ lệch 1 ✓ ⇒ **AVL**.
- (b): tại 30: h(trái) = 3, h(phải) = 1 ⇒ lệch 2 ✗ ⇒ **không** AVL.
- (c): tại 30: h(trái) = 2, h(phải) = 0 ✗ ⇒ không AVL (cây suy biến).

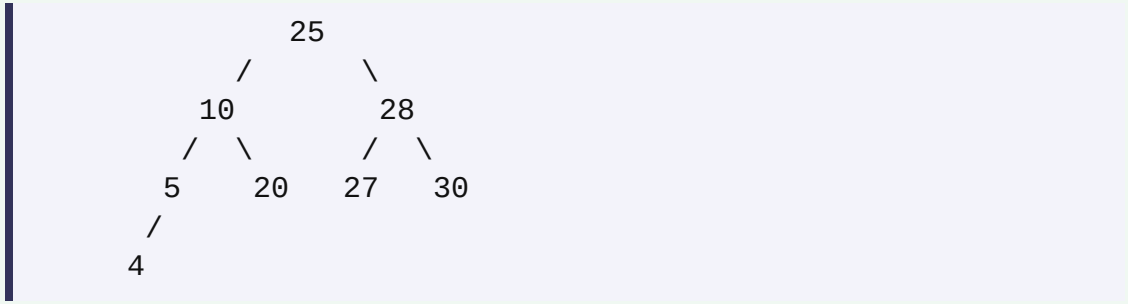
- Mọi cây cân bằng hoàn toàn đều là AVL; điều ngược lại không đúng.

1.2 Chỉ số cân bằng và chiều cao cây AVL

Chỉ số cân bằng

Bal(p) = h(cây con trái của p) − h(cây con phải của p). Cây là AVL ⇔ mọi nút có Bal ∈ {**−1, 0, +1**}. (Giáo trình ký hiệu ba trạng thái: **lệch trái** (LH), **cân bằng** (EH), **lệch phải** (RH).)

Ví dụ 12.2 – Tính Bal trên cây



Nút	h(trái)	h(phải)	Bal	Trạng thái
25	3	2	+1	Lệch trái
10	2	1	+1	Lệch trái
5	1	0	+1	Lệch trái
28	1	1	0	Cân bằng
4, 20, 27, 30	0	0	0	Cân bằng (lá)

- Mọi |Bal| ≤ 1 ⇒ **AVL**.

- **Định lý (Adelson-Velsky – Landis):** cây AVL có N nút có chiều cao **h < 1,44·log₂(N + 2)** ⇒ tìm kiếm, thêm, hủy luôn **O(log N)** kể cả trường hợp xấu nhất.
- **Ý tưởng chứng minh:** gọi N(h) là số nút **ít nhất** của cây AVL cao h: N(h) = N(h−1) + N(h−2) + 1 (giống dãy Fibonacci) ⇒ N(h) tăng theo cấp số mũ ≈ 1,618^h ⇒ h = O(log N).

N	BST suy biến	AVL (tối đa)	Cân bằng hoàn toàn
1.000	1.000	≤ 14	10
10^6	10^6	≤ 28	20

- Thực nghiệm: thêm 1, 2, …, 1000 theo thứ tự tăng vào AVL ⇒ chiều cao chỉ **10**.

1.3 Cấu trúc dữ liệu cây AVL

```
typedef struct AVL_Node
{  T Key;
   int Height;                 // chiều cao cây con gốc tại
nút
   AVL_Node *AVL_Left;
   AVL_Node *AVL_Right;
} AVL_OneNode;
typedef AVL_OneNode *AVL_Type;
int H(AVL_Type p)   { return p == NULL ? 0 : p->Height; }
int Max(int a, int b) { return a > b ? a : b; }
void Update(AVL_Type p)        // cập nhật chiều cao
{  p->Height = 1 + Max(H(p->AVL_Left), H(p->AVL_Right));  }
int Bal(AVL_Type p)            // chỉ số cân bằng
{  return H(p->AVL_Left) - H(p->AVL_Right);  }
```

- **Hai cách lưu thông tin cân bằng:**

Cách	Lưu trong nút	Ưu điểm
Giáo trình	Bal ∈ {LH, EH, RH} (2 bit)	Tiết kiệm bộ nhớ
Trong slide	Height (int)	Code ngắn, dễ hiểu, dễ kiểm tra

- Cả hai đều cho O(1) để biết nút có lệch không; slide dùng Height để code gọn hơn.
- **Tìm kiếm** trên AVL **giống hệt** BST (không cần đến Height).

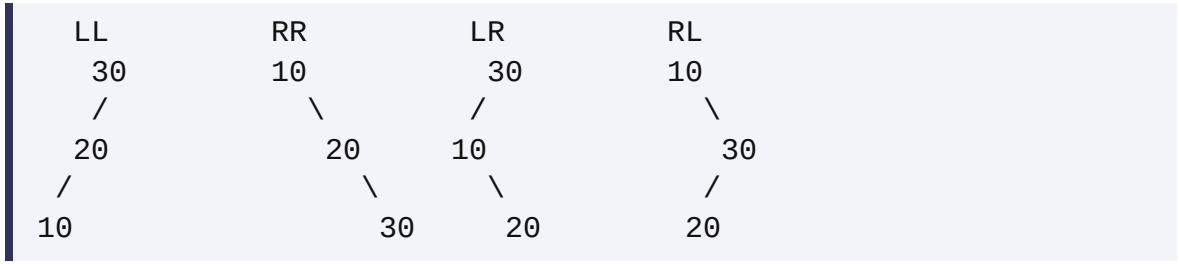
Bất biến cần giữ

Sau **mỗi** lần thêm/hủy: (1) cây vẫn là BST; (2) mọi nút có $|Bal| \le 1$; (3) Height của mọi nút đúng. Thao tác quay cây phải giữ được cả ba.

2.1 Các trường hợp mất cân bằng

- Sau khi thêm/hủy, đi ngược từ nút bị tác động lên gốc; gặp nút **P đầu tiên** có |Bal(P)| = 2 thì cân bằng lại tại P. Có 4 trường hợp, đặt tên theo đường đi từ P tới cây con bị cao:

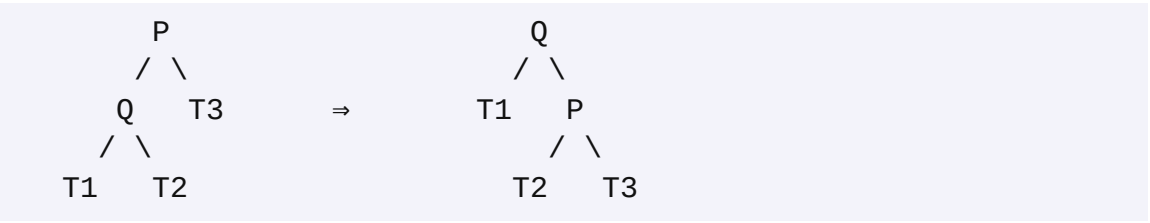
Trường hợp	Bal(P)	Con lệch	Mô tả	Cách xử lý
LL	+2	Bal(P.Left) ≥ 0	Cây con **trái** của con **trái** cao	**Quay phải** tại P
RR	−2	Bal(P.Right) ≤ 0	Cây con **phải** của con **phải** cao	**Quay trái** tại P
LR	+2	Bal(P.Left) < 0	Cây con **phải** của con **trái** cao	Quay trái tại P.Left, rồi quay phải tại P
RL	−2	Bal(P.Right) > 0	Cây con **trái** của con **phải** cao	Quay phải tại P.Right, rồi quay trái tại P



- LL, RR: **quay đơn** (1 lần). LR, RL: **quay kép** (2 lần).
- **Nhận biết nhanh:** dấu của Bal(P) và Bal(con) **cùng chiều** ⇒ quay đơn; **ngược chiều** ⇒ quay kép.
- Cả 4 ví dụ bên trái sau khi cân bằng đều thành cây **20(10, 30)**.

2.2 Quay đơn: quay phải (LL) và quay trái (RR)

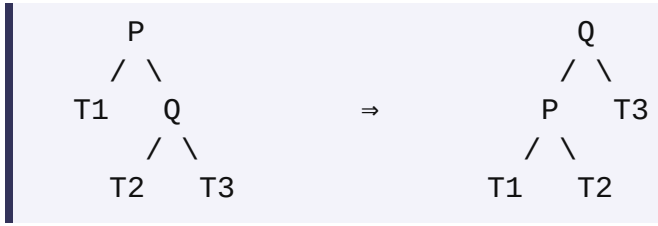
- **Quay phải tại P** (trường hợp LL): Q = con trái của P lên làm gốc; P xuống làm con phải của Q; **cây con phải của Q (T2)** chuyển sang làm con trái của P.




```
AVL_Type RotateRight(AVL_Type P)
{  AVL_Type Q = P->AVL_Left;
   P->AVL_Left = Q->AVL_Right;    // T2 sang P
   Q->AVL_Right = P;              // P xuống phải Q
   Update(P);  Update(Q);         // P trước, Q sau
   return Q;                      // gốc mới
}
```

- **Vì sao vẫn là BST?** T1 < Q < T2 < P < T3 – thứ tự LNR **không đổi**.

- **Quay trái tại P** (trường hợp RR) – đối xứng:



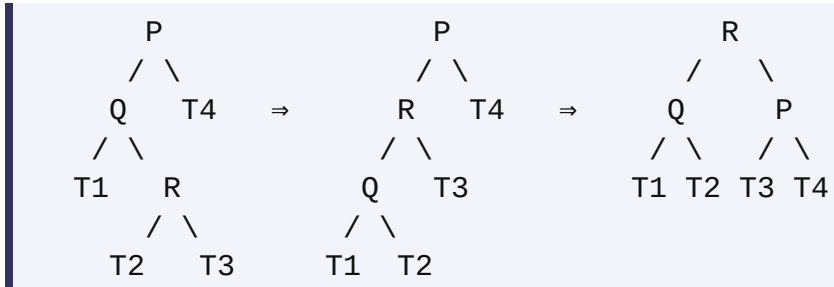

```
AVL_Type RotateLeft(AVL_Type P)
{  AVL_Type Q = P->AVL_Right;
   P->AVL_Right = Q->AVL_Left;    // T2 sang P
   Q->AVL_Left = P;               // P xuống trái Q
   Update(P);  Update(Q);
   return Q;
}
```

Ví dụ 12.3 – Thêm 10, 20, 30

Sau khi thêm 30: Bal(10) = 0 − 2 = **−2**, Bal(20) = −1 cùng chiều ⇒ **RR** ⇒ quay trái tại 10 ⇒ **20(10, 30)**. Chỉ đổi **3 con trỏ**, O(1).

2.3 Quay kép: LR và RL

- **LR:** con trái Q của P lệch **phải** (cây con R của Q cao). Quay phải tại P sẽ **không** giải quyết được (chỉ chuyển lệch sang phải). Phải:
  - B1: **quay trái tại Q** ⇒ biến thành dạng LL;
  - B2: **quay phải tại P**.



- Kết quả: **R** (cháu) lên làm gốc, Q và P thành hai con.

- **RL** đối xứng: quay phải tại P.Right, rồi quay trái tại P.
- **Hàm cân bằng chung** cho cả 4 trường hợp:

```
AVL_Type Balance(AVL_Type P)
{  Update(P);
   if (Bal(P) > 1)                       // lệch trái
   {  if (Bal(P->AVL_Left) < 0)          // LR
         P->AVL_Left = RotateLeft(P->AVL_Left);
      return RotateRight(P);             // LL
   }
   if (Bal(P) < -1)                      // lệch phải
   {  if (Bal(P->AVL_Right) > 0)         // RL
         P->AVL_Right = RotateRight(P->AVL_Right);
      return RotateLeft(P);              // RR
   }
   return P;                             // đã cân bằng
}
```

Ví dụ 12.4 – LR

Cây 30(10) thêm 20: Bal(30) = +2, Bal(10) = −1 (ngược chiều) ⇒ quay trái tại 10 ⇒ 30(20(10)) ⇒ quay phải tại 30 ⇒ **20(10, 30)**.

2.4 Thêm nút vào cây AVL

- **Thuật toán:** thêm như BST (nút mới là lá) rồi, trên đường **quay lui** từ nút mới lên gốc, **cập nhật chiều cao** và **cân bằng lại** tại nút bị lệch.

```
AVL_Type AVL_Insert(AVL_Type P, T X)
{  if (P == NULL)                          // tạo lá mới
   {  P = new AVL_OneNode;
      P->Key = X;  P->Height = 1;
      P->AVL_Left = P->AVL_Right = NULL;
      return P;
   }
   if (X < P->Key)
      P->AVL_Left = AVL_Insert(P->AVL_Left, X);
   else if (X > P->Key)
      P->AVL_Right = AVL_Insert(P->AVL_Right, X);
   else
      return P;                            // đã có X
   return Balance(P);                      // quay lui: cân
bằng
}
// Gọi: Root = AVL_Insert(Root, X);
```

- Hàm **trả về gốc mới** của cây con (vì phép quay có thể đổi gốc) ⇒ nút cha gán lại `P->AVL_Left = …`.
- **Tính chất:** khi thêm, chỉ cần **tối đa một lần cân bằng** (đơn hoặc kép); sau đó chiều cao cây con trở về như trước khi thêm ⇒ các tổ tiên phía trên không bị lệch thêm.
- **Độ phức tạp:** đi xuống O(log N) + quay lui O(log N) + quay O(1) ⇒ **O(log N)**.

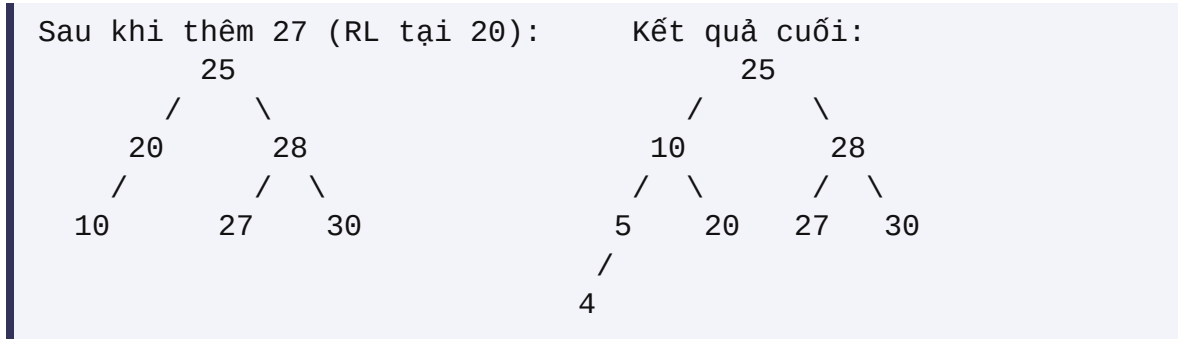
So với BST thường

Chỉ thêm **một dòng** `return Balance(P);` vào cuối hàm thêm đệ quy của BST – toàn bộ "phép màu" nằm trong hàm Balance.

2.5 Minh họa thêm dãy khóa vào cây AVL

Ví dụ 12.5 – Thêm lần lượt 10, 20, 30, 25, 28, 27, 5, 4

Thêm	Nút lệch, trường hợp	Phép quay	Cây sau khi thêm (dạng Gốc(Trái, Phải))
10	–	–	10
20	–	–	10(., 20)
30	Tại 10: Bal = −2, con phải −1 ⇒ **RR**	Quay trái tại 10	20(10, 30)
25	–	–	20(10, 30(25, .))
28	Tại 30: Bal = +2, con trái 25 có Bal −1 ⇒ **LR**	Trái tại 25, phải tại 30	20(10, 28(25, 30))
27	Tại 20: Bal = −2, con phải 28 có Bal +1 ⇒ **RL**	Phải tại 28, trái tại 20	25(20(10, .), 28(27, 30))
5	Tại 20: Bal = +2, con trái 10 có Bal +1 ⇒ **LL**	Quay phải tại 20	25(10(5, 20), 28(27, 30))
4	–	–	25(10(5(4, .), 20), 28(27, 30))



- **Phân tích bước thêm 27:** 27 > 20 ⇒ phải; 27 > 25 … đi xuống thành con trái của 28 ⇒ quay lui: Height(28) = 2, Height(25) = 3, tại 20: h(trái) = 1, h(phải) = 3 ⇒ Bal = **−2**; con phải 25 có Bal = +1 (ngược chiều) ⇒ **RL**.
- 8 khóa, chiều cao cuối = **4**; nếu dùng BST thường với cùng thứ tự thêm thì chiều cao là **6** (nhánh 10 → 20 → 30 → 25 → 28 → 27) và cây lệch hẳn sang phải.

2.6 Hủy nút trên cây AVL

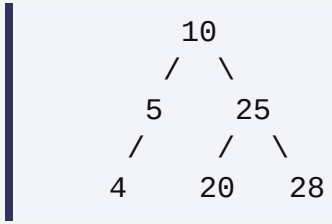
- **Thuật toán:** hủy như BST (3 trường hợp, thế mạng là trái nhất cây con phải), rồi **quay lui** cập nhật chiều cao và cân bằng lại **tại mọi nút** trên đường về gốc.

```
AVL_Type AVL_Delete(AVL_Type P, T X)
{  if (P == NULL) return NULL;            // không có X
   if (X < P->Key)
      P->AVL_Left = AVL_Delete(P->AVL_Left, X);
   else if (X > P->Key)
      P->AVL_Right = AVL_Delete(P->AVL_Right, X);
   else
   {  if (P->AVL_Left == NULL || P->AVL_Right == NULL)
      {  AVL_Type Con = P->AVL_Left ? P->AVL_Left
                                    : P->AVL_Right;
         delete P;                          // lá hoặc 1 con
         return Con;
      }
      AVL_Type M = P->AVL_Right;            // thế mạng
      while (M->AVL_Left != NULL) M = M->AVL_Left;
      P->Key = M->Key;
      P->AVL_Right = AVL_Delete(P->AVL_Right, M->Key);
   }
   return Balance(P);
}
```

- **Khác với thêm:** sau một phép quay khi hủy, chiều cao cây con có thể **giảm 1** ⇒ tổ tiên phía trên có thể lại mất cân bằng ⇒ có thể phải quay **nhiều lần** (tối đa O(log N) lần) – hàm đệ quy tự xử lý khi quay lui.
- Độ phức tạp vẫn **O(log N)**.

Ví dụ 12.6 – Từ cây cuối ví dụ 12.5, hủy 27 rồi 30

- Hủy 27 (lá): 28(., 30), Bal(28) = −1; Bal(25) = 3 − 2 = +1 ⇒ vẫn AVL.
- Hủy 30 (lá): 28 thành lá; tại 25: h(trái) = 3, h(phải) = 1 ⇒ Bal = **+2**; con trái 10 có Bal = +1 ⇒ **LL** ⇒ quay phải tại 25:



- Hủy tiếp 10 (2 con): thế mạng 20 ⇒ **20(5(4), 25(., 28))** – vẫn AVL.

2.7 Đánh giá cây AVL

Tiêu chí	BST thường	AVL
Tìm kiếm – xấu nhất	O(N)	**O(log N)**
Thêm – xấu nhất	O(N)	**O(log N)**, tối đa 1 lần cân bằng
Hủy – xấu nhất	O(N)	**O(log N)**, có thể nhiều lần quay
Bộ nhớ mỗi nút	Key + 2 con trỏ	+ Height hoặc Bal
Độ phức tạp cài đặt	Đơn giản	Phức tạp hơn (4 trường hợp quay)

- **Ưu điểm:** bảo đảm hiệu năng trong **mọi** trường hợp, không phụ thuộc thứ tự dữ liệu đầu vào; duyệt LNR vẫn cho dãy tăng.
- **Nhược điểm:** cài đặt phức tạp, tốn chi phí cân bằng khi thêm/hủy liên tục.

- **Các cây cân bằng khác:** cây **đỏ – đen** (ít quay hơn khi cập nhật, dùng trong `std::map`), **B-cây / B+-cây** (mỗi nút nhiều khóa, dùng cho chỉ mục CSDL và hệ thống tập tin), cây **Splay**, **Treap**.
- **Khi nào dùng AVL?** Tìm kiếm **nhiều hơn hẳn** cập nhật, cần bảo đảm thời gian xấu nhất (từ điển, bảng tra cứu).

Ví dụ 12.7 – Thí nghiệm: thêm 1, 2, …, N theo thứ tự tăng

N	Chiều cao BST	Chiều cao AVL	$\log_2 N$
1.000	1.000	10	≈ 10
100.000	100.000 (thường tràn stack đệ quy)	17	≈ 17

3.1 Hệ thống lại các cấu trúc dữ liệu đã học

CTDL	Tổ chức	Truy cập thứ i	Tìm	Thêm / hủy	Ưu điểm nổi bật	Buổi
Mảng / danh sách đặc	Liên tiếp	O(1)	O(N); O(log N) nếu có thứ tự	O(N) (giữa)	Truy cập nhanh, gọn	6
DSLK đơn	Nút + 1 con trỏ	O(N)	O(N)	O(1) tại vị trí đã biết	Linh hoạt kích thước	7
DSLK kép	Nút + 2 con trỏ	O(N)	O(N)	O(1); hủy cuối O(1)	Duyệt 2 chiều	8
Stack	LIFO	Chỉ đỉnh	–	Push/Pop O(1)	Đệ quy, biểu thức, Undo	9
Queue	FIFO	Chỉ đầu	–	Add/Get O(1)	Hàng chờ, BFS	9
Heap	Cây hoàn chỉnh trên mảng	Max O(1)	O(N)	O(log N)	Hàng đợi ưu tiên, HeapSort	4
Cây nhị phân	Nút + 2 con	–	O(N)	–	Phân cấp, biểu thức	10
BST	Trái < gốc < phải	–	O(h)	O(h)	Tìm + cập nhật + duyệt có thứ tự	11
AVL	BST cân bằng	–	O(log N)	O(log N)	Bảo đảm xấu nhất	12
Tập tin + chỉ mục	Bộ nhớ ngoài	fseek	O(log N) qua chỉ mục	Cập nhật chỉ mục	Dữ liệu lớn	2, 5

3.2 Hệ thống lại các giải thuật đã học

Nhóm	Giải thuật	Độ phức tạp
Tìm kiếm	Tuyến tính (cầm canh)	O(N)
	Nhị phân	O(log N)
	Theo chỉ mục (tập tin)	O(log N) lần đọc chỉ mục + 1
	Trên BST / AVL	O(h) / O(log N)
Sắp xếp $O(N^2)$	Interchange, Bubble, Shaker, Selection, Insertion	$O(N^2)$
Sắp xếp nhanh	Shell	≈ O(N^1,5)
	QuickSort	TB O(N log N), xấu $O(N^2)$
	HeapSort, MergeSort	O(N log N)
Sắp xếp ngoại	Trộn trực tiếp / tự nhiên trên tập tin; theo chỉ mục	$\log_2 N$ pass
Duyệt	Danh sách; cây NLR, LNR, LRN; theo mức	O(N)

- **Các kỹ thuật thiết kế giải thuật xuất hiện trong môn học:**
  - **Chia để trị:** tìm nhị phân, QuickSort, MergeSort, các thao tác đệ quy trên cây.
  - **Phần tử cầm canh:** tìm tuyến tính, QuickSort (X chặn vòng lặp).
  - **Tăng dần (incremental):** Insertion Sort, thêm có thứ tự.
  - **Dùng CTDL phụ để tăng tốc:** heap (HeapSort), stack (khử đệ quy), queue (duyệt theo mức), chỉ mục (tìm/sắp ngoại).
  - **Cân bằng:** AVL – đổi chi phí cập nhật lấy bảo đảm thời gian tìm.

Ví dụ 12.8 – Cùng câu hỏi "tìm X", năm câu trả lời

Mảng không thứ tự O(N) → mảng có thứ tự O(log N) → DSLK O(N) → BST O(h) → AVL O(log N): **CTDL quyết định giải thuật** và hiệu năng (CTDL + GT = CT, buổi 1).

3.3 Tiêu chí lựa chọn cấu trúc dữ liệu

- **Câu hỏi cần trả lời khi chọn CTDL:**
  - Thao tác nào **thường xuyên nhất**? (tìm, thêm, xóa, duyệt có thứ tự, lấy min/max…)
  - Kích thước dữ liệu bao nhiêu, có **biết trước** không, có vừa **bộ nhớ trong** không?
  - Có cần **thứ tự** (theo khóa, theo thời gian đến) không?
  - Yêu cầu bảo đảm thời gian **xấu nhất** hay chỉ cần trung bình tốt?
  - Độ phức tạp cài đặt, khả năng bảo trì?

Tình huống	CTDL đề xuất	Lý do
Bảng điểm lớp 50 SV, tra theo STT	Mảng	N nhỏ, truy cập theo chỉ số
Danh bạ 1 triệu số, tra cứu liên tục, ít thêm	Mảng có thứ tự + tìm nhị phân	O(log N), đơn giản
Từ điển thêm/xóa/tra liên tục, in theo vần	AVL	O(log N) mọi thao tác + LNR
Hàng đợi in ấn, tổng đài	Queue	FIFO
Kiểm tra cú pháp, Undo, tính biểu thức	Stack	LIFO
Danh sách nhạc có bài trước/sau	DSLK kép	Duyệt 2 chiều, xóa O(1)
Phòng cấp cứu – ưu tiên theo mức độ	Heap (hàng đợi ưu tiên)	Lấy max O(log N)
Sắp xếp 50 GB dữ liệu	Trộn trên tập tin	Không vừa RAM, truy cập tuần tự

3.4 Thực hành buổi 12 – Mini-project và báo cáo so sánh hiệu năng

Mini-project (chọn một đề tài, làm theo nhóm 2–3 SV)

- **Đề tài gợi ý:** quản lý thư viện; quản lý bán hàng; từ điển Anh–Việt; quản lý điểm SV; mô phỏng hàng đợi ngân hàng; quản lý danh bạ điện thoại.
- **Yêu cầu tối thiểu:**
  - Dữ liệu đọc/ghi từ **tập tin**; tối thiểu 1.000 bản ghi (sinh ngẫu nhiên nếu cần).
  - Sử dụng ít nhất **hai** CTDL khác nhau (ví dụ AVL để tra cứu + DSLK/Queue cho nghiệp vụ) và **giải thích** lý do chọn.
  - Chức năng: thêm, sửa, xóa, tìm kiếm, sắp xếp (≥ 2 thuật toán), thống kê.

Báo cáo so sánh hiệu năng

- Cài cùng chức năng "tra cứu theo khóa" trên **4 CTDL**: mảng không thứ tự, mảng có thứ tự, BST, AVL.
- Đo với N = 10^3, 10^4, 10^5: thời gian **tạo** (dữ liệu ngẫu nhiên và dữ liệu đã sắp), thời gian **10.000 lần tra cứu**, **chiều cao** cây.
- Lập bảng + biểu đồ; kết luận đối chiếu với lý thuyết (BST suy biến khi dữ liệu đã sắp; AVL ổn định).
- **Nộp:** mã nguồn, báo cáo (≤ 10 trang), trình bày 10 phút.

3.5 Câu hỏi và bài tập ôn tập tổng hợp

Câu hỏi và bài tập AVL (có đáp số)

- 1) Thêm lần lượt **50, 40, 30, 45, 47, 60** vào cây AVL rỗng; ghi rõ trường hợp mất cân bằng và phép quay. → 30: **LL** tại 50 ⇒ 40(30, 50); 45, 47: **LR** tại 50 ⇒ 40(30, 47(45, 50)); 60: **RR** tại 40 ⇒ **47(40(30, 45), 50(., 60))**.
- 2) Chứng minh phép quay phải giữ nguyên thứ tự LNR.
- 3) Vì sao khi thêm chỉ cần tối đa một lần cân bằng, còn khi hủy có thể cần nhiều lần?

Đề ôn tập tổng hợp

- 1) Cho dãy {35, 12, 47, 8, 20, 50, 3}: (a) mô phỏng Bubble Sort, QuickSort, HeapSort; (b) tìm nhị phân X = 20 trên dãy đã sắp; (c) tạo BST và AVL từ dãy, so sánh chiều cao.
- 2) Chuyển `(A + B) * C - D / (E + F)` sang hậu tố; vẽ cây biểu thức; viết NLR của cây.
- 3) Viết hàm trên DSLK đơn: xóa mọi nút có Key chẵn; chèn X vào danh sách có thứ tự; đảo ngược danh sách.
- 4) Thiết kế CTDL cho hệ thống quản lý bãi giữ xe (xe vào/ra, tra biển số, thống kê theo giờ); giải thích lựa chọn và độ phức tạp các thao tác.